




```
export function mountD3(host: HTMLElement) {  
  const w = 600, h = 300, m = 32;
```

```
  const x = d3.scalePoint()  
    .domain(labels).range([m, w - m]);  
  const y = d3.scaleLinear()  
    .domain([0, Math.max(...data)]).range([h - m, m]);  
  const line = d3.line<number>().x((_, i) => x(labels[i])).y((d) => y(d));
```

```
  const svg = d3.select(host)  
    .append("svg").attr("viewBox", `0 0 ${w} ${h}`);
```

```
  svg  
    .append("path")  
    .attr("fill", "none")  
    .attr("stroke", "#2563eb")  
    .attr("stroke-width", 2)  
    .attr("d", line(data));
```

```
  svg  
    .append("g")  
    .attr("transform", `translate(0, ${h - m})`)  
    .call(d3.axisBottom(x));
```

```
  svg  
    .append("g")  
    .attr("transform", `translate(${m}, 0)`)  
    .call(d3.axisLeft(y));  
}
```

```
export function mountPlot(host: HTMLElement) {
```

```
  const chart = Plot.plot({  
    marks: [Plot.lineY(points, {  
      x: "label", y: "value", stroke: "#2563eb"  
    })],  
  });
```

```
  host.append(chart);
```

```
}
```

D3

OldservervaldePlot



















































• append("path")

• `attn("fil", "none")`

svd

```
export function mountD3(host: HTMLDivElement) {
```

```
const x = d3.selectAll().print()
```

```
domain(labels).range([m,w-m]);
```

```
const line = d3.line<number>().x(_ => x(labels[i])!).y(d) => y(d);
```

```
domain([0, Math.max(...data)].range([h-m, m]);
```



```
const w = 600, h = 300, m = 32;
```

const y = d3.sealLinear()

```
.append("svg").attr("viewBox", 0 0 ${w} ${h});
```

const svg = d3.select(host)

```
.attr('id', 'nine(data)');
```



datastr('stroke', '#2563elb')



attr('stroke-width', 2)

```
attr('transform', translate(0, ${h-m}))
```

·call(d3·axist(y));

call(d3.axisBottom(x));

• `attr('transform', translate($n, 0))`

const chart = Plot.plot({

```
marks: [Plot.LineY(points, {
```

```
export function mountP7Lot(host: HTMLElement) {}
```






```
host.append(chart);
```



x: "label", y: "value", stroke: "#2563eb"

In my dream, there's a ladder of abstraction. On the low-level end for constructing



OpenGL Shader, say. At the highest, visual interface



Future of datawork: Q&A with Mike Bostock

November 19, 2021

for data with overview, zoom and filter, details on demand...



R















































